AI LEGAL GOVERNANCE INTELLIGENCE PLATFORM

Documentation Technique Complète

Système RAG appliqué au droit européen — GDPR, AI Act, DGA

Composant	Valeur
Stack	Python 3.12 · FastAPI · FAISS · Claude Sonnet 4
Documents	8 PDFs légaux EU (GDPR, AI Act, DGA, CNIL, EDPB)
Vecteurs	2016 chunks · 384 dimensions · all-MiniLM-L6-v2
Accélération	Apple Silicon M4 · MPS · TRANSFORMERS_OFFLINE=1
Repo	github.com/gael-cvc/AI-Legal-Governance-Platform
Version	v1.0 · Mars 2026

[00] Architecture & Choix de Stack

Avant d'écrire la première ligne de code, nous avons posé la question fondamentale : quel est le problème réel ? Les juristes, compliance officers et DPO passent des heures à naviguer dans des textes réglementaires denses. L'objectif est de construire un système capable de répondre à des questions juridiques précises en citant les sources exactes, sans hallucinations.

> **APPROCHE CABINET CONSEIL**
> Un système RAG est la réponse adaptée : il ne « génère » pas de droit, il retrouve les passages pertinents et les présente via un LLM. L'exactitude prime sur la créativité — radicalement différent d'un chatbot généraliste.

Stack technique — Comparatif décisionnel

Composant	Choix retenu	Alternative écartée	Justification
LLM	Claude Sonnet 4	GPT-4o, Mistral	Fenêtre contexte longue, raisonnement juridique, API stable
Embeddings	all-MiniLM-L6-v2	bge-large-en-v1.5	90MB, 139 chunks/sec, local, suffisant pour 2016 vecteurs
Vector Store	FAISS IndexFlatIP	Pinecone, Chroma	Exact search, 0 dépendance cloud, < 5ms sur 2016 vecteurs
Reranker	ms-marco-MiniLM-L-6-v2	Cohere Rerank	67MB, local, cross-encoder précis, gratuit
API	FastAPI	Flask, Django REST	Async natif, OpenAPI auto-généré, lifespan hooks singletons
PDF parsing	PyMuPDF	pdfplumber, pypdf2	Le plus rapide, meilleure extraction PDFs légaux EU
Accélération	MPS (Apple M4)	CPU, CUDA	3-5x plus rapide que CPU pour sentence-transformers

Infrastructure par contexte d'usage

Notre projet — Prototype local (Mac M4 · venv)

FAISS local, modèles HuggingFace en cache, FastAPI en développement. Zéro coût cloud. Idéal pour valider le concept et construire le MVP.

Cabinet Conseil Pro — Cloud managé

Pinecone ou Weaviate pour le vector store, Cohere Rerank API, déploiement Cloud Run. SLA, monitoring, authentification OAuth2. Coût estimé : ~$500/mois pour 10K req/jour.

Large Scale Entreprise — Kubernetes multi-tenant

Elasticsearch + dense vector, modèles servis via TorchServe ou Triton, pipeline Kafka pour ingestion temps réel, Grafana/Prometheus pour observabilité. Budget infra : $5K-$20K/mois selon volume.

[01] Data Lake — Architecture 3 Couches

La première décision d'architecture est la plus importante : comment organiser les données sources ? Une erreur ici compromet tout le pipeline. Nous avons adopté le pattern Medallion (raw/bronze/silver) standard en data engineering.

```
data/
├── raw/            ← JAMAIS modifié — PDFs originaux
├── bronze/         ← segments + metadata (JSONL)
├── silver/         ← chunks prêts à vectoriser (2016 lignes)
└── vector_store/   ← index FAISS + mapping JSON
```

POURQUOI LE PATTERN MEDALLION ?

La couche raw/ est immuable. Si le parser change, on re-run depuis raw. Si les chunks changent, on re-run depuis bronze. Chaque couche est reconstruite indépendamment. C'est une garantie de reproductibilité totale.

JSONL vs JSON

Lecture ligne par ligne sans charger tout en mémoire, compatibilité Spark/pandas/jq, append atomique sans réécrire tout le fichier. Choix systématique pour tout pipeline data.

Corpus légal EU — 8 documents sources

Document	Type	Mode parsing	Particularité
gdpr_full.pdf	Règlement EN	Article extraction	Articles 1-99
rgpd_full.pdf	Règlement FR	Article extraction	Articles 1-99 (bilingue)
rgpd_recitals.pdf	Recitals FR	Recital extraction	Recitals 1-173
ai_act_full.pdf	Règlement	Article extraction	Articles 1-113
ai_act_annexes.pdf	Annexes	Annex extraction	Annexes I-XIII
data_governance_act.pdf	Règlement	Article extraction	Articles 1-36
edpb_automated_decision.pdf	Guidelines	Freetext (1 page = 1 segment)	Pas de structure Article X
cnil_ai_recommendations.pdf	Guidelines	Freetext (1 page = 1 segment)	176 pages, prose continue

CAS PARTICULIER — CNIL & EDPB

Ces documents n'ont pas de structure 'Article X'. Le parser détecte l'absence d'ancres et bascule automatiquement en mode freetext. Fallback robuste qui préserve le contexte sans forcer une structure inexistante.

[02] Étape 1 — Pipeline d'Ingestion

Architecture des modules

Module	Rôle	Input → Output
pdf_parser.py	Extraction brute + nettoyage bruit	PDF → texte propre
article_extractor.py	Détection ancres + slicing	texte → segments structurés
metadata_builder.py	Enrichissement métadonnées	segments → segments + metadata
chunker.py	Découpage sémantique avec overlap	segments → chunks JSONL
pipeline.py	Orchestration raw → silver	data/raw/ → data/silver/

Paramètres de chunking — Raisonnement détaillé

max_chars = 1200 (~300 tokens)

all-MiniLM-L6-v2 a une limite de 256 tokens. On reste sous ce seuil pour garantir que l'embedding capture le chunk entier. 1200 chars = ~300 tokens = marge de sécurité de 15%. Au-delà, le modèle tronque silencieusement.

overlap = 200 chars (17%)

Standard RAG : 10-20% d'overlap. Un Article GDPR peut se terminer en milieu de phrase — le chunk N+1 reprend les 200 derniers caractères pour préserver le contexte légal complet. Sans overlap, les définitions à cheval sur deux chunks sont perdues.

RÉSULTATS / LOGS

```
$ python -m ingestion.pipeline

[✓] gdpr_full.pdf          → 99 articles  → 387 chunks
[✓] rgpd_full.pdf          → 99 articles  → 391 chunks
[✓] rgpd_recitals.pdf      → 173 recitals → 298 chunks
[✓] ai_act_full.pdf        → 113 articles → 441 chunks
[✓] ai_act_annexes.pdf     → 13 annexes   → 89 chunks
[✓] data_governance_act.pdf→ 36 articles  → 134 chunks
[✓] edpb_automated_decision.pdf → 37 pages → 148 chunks
[✓] cnil_ai_recommendations.pdf → 176 pages→ 128 chunks

8/8 documents traités ✓
Bronze : 1054 segments   Silver : 2016 chunks
Durée totale : ~45 secondes
```

[03] Étape 2 — Embedding & Index FAISS

L'embedding transforme chaque chunk texte en vecteur numérique dense. Deux chunks sémantiquement proches auront des vecteurs proches dans l'espace vectoriel. C'est la fondation de la recherche sémantique — radicalement différent d'une recherche par mots-clés.

Comparatif des modèles d'embedding

Modèle	Dim.	Tokens	Taille	Vitesse M4	Verdict
all-MiniLM-L6-v2 ★	384	256	90 MB	139/sec	RETENU
bge-large-en-v1.5	1024	512	1.3 GB	~40/sec	Trop lent/lourd
text-embedding-ada-002	1536	8191	API	Réseau	Coût + latence
multilingual-e5-large	1024	512	1.1 GB	~35/sec	Si multi-langue critique

RAISONNEMENT — Pourquoi all-MiniLM-L6-v2

Sur 2016 vecteurs, la qualité d'embedding est moins critique que sur 2M vecteurs. all-MiniLM atteint 58.8 BEIR vs 63.6 pour bge-large — 8% de différence pour 3x le coût en vitesse. Le reranker (étape 4) compense ce delta. Trade-off calculé et documenté.

FAISS — Choix de l'index

IndexFlatIP (notre choix) — Recherche exacte par Inner Product. On normalise en L2 → IP = cosine similarity. Aucune approximation, résultats garantis optimaux. Latence < 5ms sur 2016 vecteurs.

IndexIVFFlat (alternative scale) — Divise l'espace en clusters. 10-100x plus rapide sur >100K vecteurs mais résultats approximatifs. Indispensable au-delà de 500K vecteurs.

```
# Logique core build_index.py
embedder = SentenceTransformer('all-MiniLM-L6-v2', device='mps')
embeddings = embedder.encode(texts, show_progress_bar=False)
# ⚠️ show_progress_bar=False obligatoire en contexte async

faiss.normalize_L2(embeddings)  # cosine via IndexFlatIP
index = faiss.IndexFlatIP(384)  # 384 = dimension
index.add(embeddings)           # 2016 vecteurs
faiss.write_index(index, 'data/vector_store/legal_index.faiss')
```

RÉSULTATS / LOGS

```
$ python -m rag.build_index

Chargement de 2016 chunks depuis silver/...
```

```
Encodage sur device: mps
Vitesse : 139 chunks/sec
Durée encodage : ~14.5 secondes
Normalisation L2 : OK
Index FAISS : 2016 vecteurs, 384 dimensions
Taille sur disque : 3024 KB (legal_index.faiss)
[✓] Index construit avec succès

UserWarning: UNEXPECTED: embeddings.position_ids
→ Warning normal de compatibilité HuggingFace, pas une erreur
```

BUG RÉSOLU — Déduplication

La query expansion causait l'apparition du même Recital 26 deux fois dans les résultats (scores légèrement différents). Fix : ajout d'un set seen_segments dans search() — vérification via segment_id avant chaque ajout. O(1) par lookup, aucun impact performance.

[04] Étape 3 — API FastAPI

FastAPI est le seul framework Python qui offre async natif, génération automatique OpenAPI/Swagger, et lifespan hooks — essentiels pour charger les modèles ML une seule fois au démarrage sans bloquer les requêtes.

Pattern Singleton — Architecture critique

> **DÉCISION ARCHITECTURALE MAJEURE**
> Les modèles ML sont lourds : all-MiniLM prend ~2s, le cross-encoder ~3s. Sans singleton, chaque requête HTTP rechargerait les modèles → latence insupportable. Le lifespan() charge tout une seule fois au démarrage et garde les instances en mémoire pour toute la durée de vie du serveur.

```
# 3 singletons globaux chargés une seule fois
_vector_store: VectorStore | None = None
_embedder:     LegalEmbedder | None = None
_reranker:     LegalReranker | None = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    # STARTUP
    _vector_store = VectorStore(); _vector_store.load()
    _embedder     = LegalEmbedder()
    _reranker     = LegalReranker()  # lazy load
    yield
    # SHUTDOWN — libération mémoire

# Getters thread-safe
def get_vector_store() -> VectorStore: return _vector_store
def get_reranker()     -> LegalReranker: return _reranker
```

Pipeline de recherche — 4 étapes

Étape	Composant	Description
1	Query Expansion	Claude génère 3 reformulations → FAISS cherche chacune → résultats fusionnés et dédupliqués
2	FAISS Retrieval	Récupère k*2 candidats (si k=3 → 6 résultats) pour alimenter le reranker
3	Reranking optionnel	Si use_reranking=true, cross-encoder reclasse les candidats par pertinence réelle
4	Génération Claude	Top-k chunks injectés dans le contexte. Instruction stricte : citer sources, refuser si absent

Lancement & Configuration

```
# Variable d'env obligatoire — modèles en cache local
TRANSFORMERS_OFFLINE=1 uvicorn api.main:app --reload --port 8000
```

```
# TRANSFORMERS_OFFLINE=1 : évite les timeouts HuggingFace Hub
# Sans cette variable : 30s de timeout à chaque démarrage

# Swagger UI : http://localhost:8000/docs
# Health check :
curl http://localhost:8000/health
{"status":"ok","vector_store":true,"embedder":true,"reranker":true}
```

[05] Étape 4 — Cross-Encoder Reranker

Le reranker répond à un problème fondamental : les bi-encoders (FAISS) sont rapides mais imprécis. Le cross-encoder est lent mais d'une précision remarquable. On combine les deux en pipeline two-stage.

	Bi-encoder (FAISS)	Cross-encoder (Reranker)
Méthode	Encode query + docs séparément	Encode [query + doc] ensemble
Vitesse	< 5ms (constant)	~1-2s pour 6 paires
Précision	Bonne (cosine similarity)	Excellente (interactions contextuelles)
Scalabilité	O(1) — millions de docs	O(n) — max quelques centaines
Usage	Stage 1 — filtrer les candidats	Stage 2 — reclasser les finalistes

RAISONNEMENT TWO-STAGE

FAISS récupère k*2=6 candidats en < 5ms. Le cross-encoder reclasse ces 6 en ~1-2s. Si on avait appliqué le cross-encoder sur les 2016 vecteurs directement, cela prendrait ~75 minutes. Le two-stage permet la précision du cross-encoder avec la scalabilité du bi-encoder.

```
class LegalReranker:
    MODEL = 'cross-encoder/ms-marco-MiniLM-L-6-v2'  # 67MB

    def __init__(self):
        self._model = None  # lazy load

    def _load(self):
        if self._model is None:
            self._model = CrossEncoder(self.MODEL, device='mps')
            # ⚠️ device='mps' obligatoire sur M4

    def rerank(self, query, chunks, top_k):
        self._load()
        pairs = [(query, c['text']) for c in chunks]
        scores = self._model.predict(pairs,
                     show_progress_bar=False)  # ⚠️ async safe
        ranked = sorted(zip(chunks, scores),
                        key=lambda x: x[1], reverse=True)
        return [c | {'rerank_score': float(s)} for c, s in ranked[:top_k]]
```

Résultats de validation — Analyse détaillée

RÉSULTATS / LOGS

```
Query: "What are the obligations of a data controller under GDPR?"
k=3, use_reranking=true → FAISS récupère k*2=6 candidats

Scores FAISS (bi-encoder) :
  1. Recital 95   score: 0.661  ← FAISS top 1
  2. Recital 79   score: 0.659
  3. Recital 74   score: 0.647  ← FAISS top 3
```

```
Scores Cross-Encoder (après reranking) :
  1. Recital 74   rerank: 3.42  ← PROMU de #3 à #1
  2. Recital 79   rerank: 2.11  ← stable
  3. Recital 95   rerank: 1.03  ← RÉTROGRADÉ de #1 à #3

Interprétation : Recital 74 = 'responsabilité du responsable du traitement'
Le cross-encoder a correctement détecté la correspondance sémantique
profonde entre 'obligations data controller' et Recital 74.
Le FAISS l'avait sous-estimé : moins de tokens en commun en surface.

Modèle : claude-sonnet-4-20250514
Temps total : 14.3s ✓
```

[06] Git — Versioning & Workflow

Le versioning n'est pas une contrainte administrative — c'est un outil de traçabilité des décisions techniques. Chaque commit documente quoi a changé, pourquoi, et dans quel contexte.

Setup initial

```
git init
git config user.name "Gaël"
# .gitignore — éléments critiques :
.env                  # ← CRITIQUE : contient ANTHROPIC_API_KEY
venv/                 # dépendances Python non versionnées
__pycache__/          # bytecode Python
data/raw/             # PDFs sources (binaires lourds)
data/vector_store/    # index FAISS généré (reproductible)

git add .
git commit -m "chore: initial project structure"
git remote add origin git@github.com:gael-cvc/AI-Legal-Governance-Platform.git
git push -u origin main
```

Workflow feature/reranking — Cycle complet

```
# Créer la branche depuis main
git checkout -b feature/reranking

# Commits atomiques pendant le développement
git add rag/reranker.py
git commit -m "feat(rag): add LegalReranker with cross-encoder singleton"
git add api/search.py api/main.py
git commit -m "feat(api): integrate reranker into search pipeline"

# Merge fast-forward (historique linéaire)
git checkout main
git merge --ff-only feature/reranking
# Fast-forward — 3 files changed: api/main.py, api/search.py, rag/reranker.py

# Nettoyer la branche
git branch -d feature/reranking
git push origin main
```

> **POURQUOI --ff-only ?**
> Garantit que le merge ne crée pas de 'merge commit' parasite. Si la branche a divergé, le merge échoue proprement. L'historique reste linéaire et lisible avec git log --oneline. Standard en équipes pro.

> **PROBLÈME SSH RÉSOLU — Port 443**
> Le port 22 (SSH standard) était bloqué par le réseau. Solution : ~/.ssh/config → Host github.com / Hostname ssh.github.com / Port 443. Git utilise alors le port HTTPS pour tunneler SSH.

Convention de commits

Type	Usage	Exemple
feat	Nouvelle fonctionnalité	feat(reranking): cross-encoder singleton
fix	Correction de bug	fix(rag): deduplicate via seen_segments
chore	Maintenance, config	chore: initial project structure
docs	Documentation	docs: add README setup
perf	Optimisation	perf(embedder): switch device cpu → mps

[07] Problèmes Rencontrés & Solutions

Tout projet réel rencontre des obstacles. La valeur d'un cabinet conseil réside dans sa capacité à diagnostiquer rapidement, décider rationnellement, et documenter pour que les mêmes erreurs ne se reproduisent pas.

#	Symptôme	Cause	Solution	Alternative
01	Fichiers Python créés vides dans VS Code	VS Code crée des placeholders avant copie du contenu	Créer via terminal (touch + nano) ou drag-drop des fichiers téléchargés	Script de scaffolding automatique
02	__pycache__ cachant les anciens fichiers vides	Python compile .pyc depuis les fichiers sources vides	rm -rf **/__pycache__	pytest --cache-clear à chaque run
03	device='cpu' lent sur M4 (~2min pour 2016 chunks)	sentence-transformers utilise CPU par défaut sur Mac	device='mps' partout — 8x plus rapide	CUDA si Linux avec GPU NVIDIA
04	progress_bar bloquant l'event loop FastAPI	tqdm écrit sur stderr de façon synchrone	show_progress_bar=False dans tous .encode() et .predict()	run_in_executor() pour isoler le thread
05	Recital 26 apparaissant deux fois dans les résultats	Query expansion : même doc dans top-k de 2 reformulations	Set seen_segments dans search() — vérification par segment_id	Fusion par score max plutôt que déduplication stricte

[08] Résultats Finaux & État du Projet

Métriques de production

Métrique	Valeur	Contexte
Documents traités	8/8 (100%)	Aucune perte, tous les modes parsés
Chunks silver	2016	Prêts à vectoriser
Dimensions vecteur	384	all-MiniLM-L6-v2
Taille index FAISS	3024 KB	Tient entièrement en RAM
Vitesse encodage	139 chunks/sec	MPS, Apple M4
Latence FAISS search	< 5ms	2016 vecteurs, exact search
Temps reranking (6 paires)	~1-2s	Cross-encoder MPS
Temps total /search	14.3s	Query expansion + FAISS + rerank + Claude
Tests validation	5/5 ✓	Résultats pertinents, sans doublons

RÉSULTATS / LOGS

```
POST /search
{"query":"What are the obligations of a data controller under GDPR?",
 "k":3,"use_reranking":true}

→ Réponse Claude (extrait) :
  Under the GDPR, a data controller bears primary responsibility
  for ensuring compliance. According to Recital 74, the controller
  must implement appropriate technical and organisational measures...

Sources : Recital 74 (3.42) | Recital 79 (2.11) | Recital 95 (1.03)
Modèle : claude-sonnet-4-20250514 | Temps : 14.3s ✓
```

État Git — main branch

```
$ git log --oneline

b7e8f2a feat(reranking): cross-encoder singleton — LegalReranker
a8d3c12 feat(api): FastAPI with singleton pattern + query expansion
9f1e5b3 fix(rag): deduplicate results via seen_segments set
7c2a8d1 feat(rag): FAISS IndexFlatIP + build_index pipeline
4e9b2f7 feat(ingestion): full pipeline raw→bronze→silver (2016 chunks)
2a5d8c4 chore: initial project structure + .gitignore

# Branche active : main
# feature/reranking : merged ✓ + deleted ✓
# Remote push : pending (SSH port 443 — réseau instable)
```

Prochaines étapes recommandées

Étape 5 — Évaluation RAG

- Construire un dataset de 50 questions/réponses de référence

- Mesurer Precision@k, Recall@k, MRR avec RAGAS ou TruLens
- Comparer reranking ON vs OFF pour quantifier le gain

Étape 6 — Interface utilisateur

- Frontend Streamlit ou Next.js avec affichage des sources
- Historique des conversations + export PDF des réponses

Étape 7 — Production

- Authentification JWT ou API keys
- Docker + déploiement Cloud Run + monitoring Grafana